

CS 527 Research Project Report: Group-20

Brandon Li, Jenny Chung
University of Illinois at Urbana-Champaign
[bcli3,jennyc3]@illinois.edu

Abstract

Flaky tests pass or fail non-deterministically across CI runs without changes to the code. These types of tests undermine software development by eroding developer trust, slowing CI/CD pipelines, and masking regressions that reach production. One prominent yet elusive category of flaky tests is implementation-dependent (ID) tests, which fail due to incorrect assumptions about under-constrained APIs (e.g., hash map iteration order) or environments (e.g., platform quirks or timing subtleties).

Much prior work focuses on Java projects, with tools emphasizing broad flakiness or order-dependent (OD) tests via test reordering. Although Python-specific flaky test studies exist, they primarily target OD flakiness. ID tests remain prevalent in Python’s dynamic standard library and third-party APIs, yet they are largely undetected and unaddressed because they require deep code introspection.

To bridge this gap, we propose a technique for automatically detecting and fixing Python ID tests. By analyzing code internals and randomizing nondeterministic APIs without requiring manual insight, our approach identifies wrong implementation assumptions and generates targeted deterministic fixes to restore test reliability.

1 Artifact Information

Link to the GitHub Repository: [put the link here]

2 Introduction

To detect bugs and regressions in software development, developers run automated tests that execute the software under test (SUT) and verify results against expected outcomes. Passing tests provide confidence when merging changes, while failing tests prompt debugging of recent modifications to identify and resolve issues. Reliable testing therefore underpins robust software development by verifying correctness amid rapid changes and enabling confident CI/CD iteration. However, flaky tests, which are tests that nondeterministically pass or fail on the same code version, can weaken this assurance. By undermining the core premise of regression testing, where failures signal real problems, flaky tests may mislead developers into chasing false alarms rather than genuine regressions.

Flaky tests are widely recognized as a major challenge in software development. They are not only problematic, but also prevalent in large codebases, where even a small percentage of flakiness can translate into substantial productivity losses. For example, a study from Google [1] found that flaky tests account for 4.56% of test failures and consume over 2% of coding time. Similar industry observations report that flaky tests increase CI reruns, debugging overhead, and reduce developer trust in automation. These hidden costs compound across the software development lifecycle. Despite growing awareness of flaky tests, certain categories such as ID tests

in Python, remain underexplored, motivating the need for targeted detection and repair techniques.

Extensive research has targeted the detection and mitigation of flaky tests, recognizing them as a major challenge to test reliability and developer productivity. Much of this work has focused on Java, where large datasets have enabled the development of sophisticated techniques for identifying and repairing flaky behavior. Approaches leverage strategies such as repeated test execution, dependency analysis, and runtime monitoring to expose nondeterministic outcomes. For instance, *iDFlakies* focuses on detecting and classifying flaky tests using test reordering and reruns [2], while *Nondex* dynamically analyzes test executions in Java to detect implementation-dependent flakiness [3]. More recent tools, such as *DexFix*, extend these approaches to automatically repair wrong assumptions on underdetermined specifications, specifically targeting ID flakiness in Java tests [4]. Together, these advances have significantly improved the understanding of flaky tests and established a strong foundation for addressing flakiness in the Java world.

In contrast, research on flaky tests in Python remains limited, despite evidence suggesting that flakiness is as prevalent in Python as in Java. One large-scale study [5] found that order dependency is a dominant problem in Python, accounting for 59% of flaky tests, while 28% stemmed from test infrastructure issues. The remaining failures were largely associated with network and randomness APIs, highlighting the role of nondeterministic behavior in Python. These findings have guided much of the existing Python literature toward order-dependent flakiness, emphasizing detection through randomized test ordering or rerun strategies, with comparatively less focus on automated repair. While existing work provides valuable insight into Python flaky behavior, they explore a narrower portion of the problem space compared to Java. Notably, ID flaky tests in Python remain largely underexplored, leaving a gap in techniques for both detecting and automatically repairing this class of failures.

Although existing work has advanced the understanding of flaky tests, a critical gap remains in addressing ID flaky tests in Python. Most techniques, including Java-centric approaches such as *DexFix*, cannot be ported to Python due to differences in runtime behavior and test infrastructure [4]. Research on Python flaky tests has primarily emphasized OD flakiness, leaving failures arising from incorrect assumptions on underdetermined APIs largely underexplored. Existing Python tools mainly prioritize detection over repair, and repair tools such as *iPFlakies* are limited to fixing OD flakiness through test reordering [6]. Large language model (LLM) efforts like *NIODebugger* target non-OD flakiness, but lack Python ID specialization [7]. These limitations cause existing techniques to miss subtle sources of nondeterminism inherent in Python’s runtime, leaving developers with little support for addressing ID flaky tests. Since no technique to date provides specialized detection and

automated repair for this class of failures, this gap calls for techniques that can reliably detect and automatically repair ID tests while preserving their original intent.

To bridge this research gap, we propose a technique for automatically detecting and fixing ID tests in Python. Repairing ID flakiness is inherently challenging because it requires reasoning about code internals and identifying subtle assumptions about runtime behavior that are often difficult to detect through traditional testing approaches. Our approach consists of four key components: a static pattern analyzer that identifies suspicious patterns in test code such as unordered collection iteration, unseeded randomness, or timestamp dependencies; an execution variant generator that runs tests under controlled environmental variations to confirm nondeterministic behavior; a deterministic patch generator that produces rule-based fixes, such as sorting unordered iterations or seeding random operations, while preserving test intent; and a determinism validator that verifies patched tests against the same environmental variations to ensure consistent outcomes. Implemented as a lightweight Pytest plugin, the proposed technique integrates into existing Python CI workflows and supports the repair of flaky ID tests without altering their semantic intent.

Our experimental methodology consists of three main stages: dataset construction, tool implementation, and systematic testing. In the dataset construction stage, we will use an LLM to generate 10-15 self-contained Python programs. For each program, we will create a pytest test suite containing both deterministic tests and intentionally flaky tests with specific known patterns (dictionary/set iteration without sorting, random calls without seeds, exact float comparisons, and filesystem operations without ordering). Each flaky test will be documented with its root cause and expected fix to serve as ground truth. In the implementation stage, we will develop the tool's four core components: a Static Pattern Analyzer using Python's AST module to detect suspicious code patterns, an Execution Variant Generator that runs tests under different PYTHONHASHSEED values and random states, a Deterministic Patch Generator that applies rule-based transformations (adding sorted(), injecting seeds, using pytest.approx(), etc.), and a Determinism Validator that re-executes patched tests to confirm fix effectiveness. In the testing stage, we will first validate the tool on our controlled dataset to establish baseline performance, then, if time permits, apply it to 2-3 real-world repositories (targeting scipy, numpy, pandas, or pytest) to test generalizability. All experiments will be conducted on Ubuntu 24.04 with Python 3.10+, and we will systematically vary PYTHONHASHSEED across at least 10 different values for variant execution. We will log all detection results, variant execution outcomes, generated patches, and validation results.

One paragraph about the results of experiments

A list of contributions your technique/paper makes.

3 Proposed Technique

3.1 Approach Overview

Our tool consists of four main components that work together to identify and repair implementation-dependent flaky tests in Python.

The first component is the Static Pattern Analyzer. This component performs static code analysis on test files to identify suspicious

patterns that may lead to nondeterministic behavior. Unlike tools that focus on test execution order, our analyzer targets inherent implementation-dependent behaviors within individual tests. The analyzer's detection patterns include unordered collection iteration, hash-dependent operations, unseeded randomness, timestamp dependencies, and set/dictionary comparison without order independence.

The second component is the Execution Variant Generator. Rather than repeatedly running the same test multiple times, which can miss implementation-dependent flakiness, this component executes tests under controlled environmental variations to expose nondeterministic behavior. Variant strategies include hash seed variation, random seed perturbation, collection implementation switching, and time mocking. Each suspicious pattern identified by the Static Pattern Analyzer is paired with appropriate execution variants. The component tracks which variants produce different outcomes, confirming the test's flakiness.

The third component is the Deterministic Patch Generator. This component generates rule-based fixes for confirmed flaky tests, focusing on making execution deterministic while preserving test intent. For instance, for a flaky test using unordered iteration, the tool will wrap the iteration in a sorted() call with appropriate key functions. Any sets will be converted to sorted lists before iteration, and dictionary iteration will be replaced with sorted(dict.items()).

The fourth component is the Determinism Validator. This component verifies that applied patches successfully eliminate flakiness by re-executing tests under the same environmental variations used during detection. The process re-runs the patched tests with multiple hash seeds, executes tests with varied random seeds if randomness was detected, confirms consistent pass/fail outcomes, and flags tests that still exhibit nondeterministic behavior for manual review.

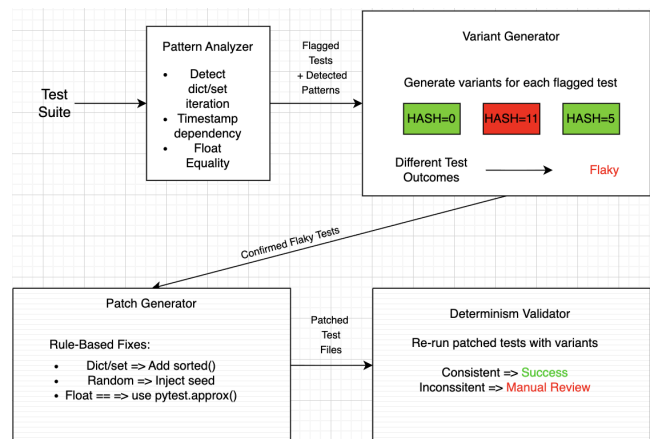


Figure 1: Flow Diagram of Tool

3.2 Approach Details

Describe the details of each component in this subsection

4 Experimental Evaluation

This work aims to answer the following research questions:

- How effectively can static analysis identify implementation-dependent flaky tests compared to traditional rerun-based approaches?
- How does variant-based execution compare to repeated reruns in confirming test flakiness?
- What percentage of detected flaky tests can be automatically repaired using rule-based transformations?
- Do the generated patches preserve test correctness while eliminating flakiness?
- How does the tool scale to large test suites in terms of analysis time and resource usage?
- How does implementation-dependent flakiness compare to order-dependent (OD) flakiness in prevalence and impact?

4.1 Evaluation Setting

To assess the effectiveness of our tool, we will conduct an evaluation using both controlled and, if time permits, real-world test suites. For phase 1, we will generate a ground-truth dataset of approximately 20-30 simple Python programs with intentionally injected flaky tests covering various flakiness patterns: unordered dictionary/set iteration, unseeded randomness, floating-point equality issues, and filesystem ordering dependencies. Each program will include a mix of deterministic baseline tests and flaky tests with known patterns and documented expected fixes. This controlled setting allows us to precisely measure detection accuracy (precision and recall), confirmation effectiveness through variant execution, repair success rates for each pattern type, and validate that patches preserve test semantics. We will compare our variant-based confirmation approach against traditional rerun-based detection (i.e. running tests 10, 100, and 1000 times) to evaluate efficiency gains.

Potentially for a phase 2, we will apply the tool to Python repositories from GitHub to assess generalizability and practical impact. Target repositories may include projects from Scientific Computing libraries like SciPy, NumPy, pandas or developer tools like pytest itself. We will prioritize repositories with active development and pytest-based test suites, reported flaky test issues in their issue trackers or commit history, and a minimum number of tests to provide sufficient data. Where possible, we will compare our automatically generated patches against human-written fixes from Git commit history to assess patch quality. Success will be measured by our ability to detect and successfully repair implementation-dependent flaky tests while maintaining low false positive rates and reasonable performance overhead.

4.2 Evaluation Results

Present the results of individual research questions in this subsection. Use tables/figures to present the results. You need to discuss your observations and speculate the reasons for observations.

5 Related Work

Our tool addresses implementation-dependent flaky test detection and repair in Python, which differs fundamentally from the dominant focus on order-dependent (OD) flaky tests in existing research.

We categorize related work into three main approaches: order-dependent test detection/repair, implementation-dependent test detection, and hybrid/machine learning approaches.

Order-Dependent Flaky Test Detection and Repair:

iPFlakies [6] is the most closely related work to our project, as it targets Python flaky tests. However, iPFlakies focuses exclusively on order-dependent tests, which pass or fail depending on the execution order of other tests. The framework detects OD tests by running test suites in randomized orders and automatically generates patches by finding “helper” tests that clean polluted shared state. Similarly, iDFlakies [2] and iFixFlakies [8] provide detection and repair for order-dependent tests in Java projects, with iFixFlakies searching for helper tests to reset state and generating patches for 58 out of 110 OD tests. ODRRepair [9] extends this approach by using automatic test generation (Randoop) to create cleaners when helper tests don’t exist in the test suite.

The prevalence of flaky tests in both research and industry motivates the need for automated detection and repair tools. Luo et al. [1] conducted one of the first comprehensive empirical studies on flaky tests, analyzing 201 commits from 51 open-source projects and establishing a widely-used taxonomy of root causes. Their study found that flaky tests are pervasive, with major tech companies reporting that flaky tests significantly impact developer productivity and CI/CD reliability.

Our tool differs fundamentally: Rather than detecting flakiness caused by test execution order, we target implementation-dependent flakiness, which fail due to nondeterministic behavior within individual tests themselves (i.e. unordered dictionary/set iteration, unseeded randomness, floating-point equality). These flaky tests fail regardless of execution order and require different detection and repair strategies.

Implementation-Dependent Flaky Test Detection:

NonDex [3, 12] is the only existing tool specifically designed to detect implementation-dependent flaky tests, but it targets Java programs. NonDex detects tests that make incorrect assumptions about underdetermined Java API specifications (e.g., assuming HashSet iteration order is deterministic) by exploring different allowed API behaviors through bytecode instrumentation. The tool has been successfully applied to detect hundreds of ID flaky tests in open-source Java projects by systematically exploring non-deterministic API behaviors allowed by the Java specification.

DexFix [4] extends NonDex by automatically generating domain-specific patches for Java, such as replacing HashMap with LinkedHashMap or adding sorting operations. DexFix represents the state-of-the-art in automated repair for implementation-dependent flaky tests in Java, successfully fixing tests that fail due to wrong assumptions about underdetermined specifications.

Our tool differs in three key ways: First, we target Python rather than Java, which has different sources of nondeterminism (PYTHONHASHSEED, different standard library behaviors). Second, we use static pattern analysis combined with variant-based execution (different hash seeds, random states) rather than bytecode instrumentation. Third, we address a broader range of implementation-dependent patterns including unseeded randomness, floating-point

equality, and filesystem ordering, not just collection iteration.

Machine Learning and Empirical Approaches:

FlakeFlagger [10] uses machine learning (ML) to predict flaky tests without rerunning, achieving 74% recall and 60% precision by analyzing behavioral features collected during test execution. CANNIER [11] combines ML predictions with selective rerunning to reduce time cost by 88% while maintaining high detection accuracy.

An empirical study by Gruber et al. [5] specifically examined flaky tests in Python, identifying 7,571 flaky tests across 1,006 Python projects. Their study found that while 59% were order-dependent, a significant portion exhibited other types of flakiness including randomness and infrastructure issues. This empirical evidence demonstrates that implementation-dependent flakiness is a substantial problem in Python projects, yet existing tools like iPFlakies focus primarily on order-dependent tests, leaving a critical gap in automated detection and repair for implementation-dependent flaky tests in Python.

Our tool differs by: (1) Using rule-based static analysis rather than ML for detection, providing deterministic and interpretable results without requiring training data; (2) Confirming flakiness through targeted variant execution rather than repeated reruns, which is more efficient for implementation-dependent patterns; (3) Providing automatic repair through deterministic transformations rather than just detection.

References

- [1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An Empirical Analysis of Flaky Tests," in *FSE*, 2014. <https://mir.cs.illinois.edu/lamyaa/publications/fse14.pdf>
- [2] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests," in *ICST*, 2019. <https://ieeexplore.ieee.org/document/8730188/>
- [3] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, "NonDex: A Tool for Detecting and Debugging Wrong Assumptions on Java API Specifications," in *FSE*, 2016. <https://github.com/TestingResearchIllinois/NonDex>
- [4] P. Zhang, Y. Jiang, A. Wei, V. Stodden, D. Marinov, and A. Shi, "Domain-Specific Fixes for Flaky Tests with Wrong Assumptions on Underdetermined Specifications," in *ICSE*, 2021. <https://ieeexplore.ieee.org/document/9402009/>
- [5] M. Gruber, S. Lukasczyk, F. Krois, and G. Fraser, "An Empirical Study of Flaky Tests in Python," in *ICST*, 2021. <https://arxiv.org/pdf/2101.09077>
- [6] R. Wang, Y. Chen, and W. Lam, "iPFlakies: A Framework for Detecting and Fixing Python Order-Dependent Flaky Tests," in *ACM SIGSOFT Software Engineering Notes*, 2022. <https://yangc9.github.io/files/WangETAL22iPFlakies.pdf>
- [7] Kaiyao Ke, "NIODebugger: A Novel Approach to Repair Non-Idempotent Outcome Tests with LLM-Based Agent," in *IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. <https://doi.org/10.1109/ICSE55347.2025.00226>
- [8] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFixFlakies: A Framework for Automatically Fixing Order-Dependent Flaky Tests," in *ESEC/FSE*, 2019. <https://dl.acm.org/doi/10.1145/3338906.3338925>
- [9] C. Li, C. Zhu, W. Wang, and A. Shi, "Repairing Order-Dependent Flaky Tests via Test Generation," in *ICSE*, 2022. <https://wenxiwang.github.io/papers/ICSE2022.pdf>
- [10] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "FlakeFlagger: Predicting Flakiness Without Rerunning Tests," in *ICSE*, 2021. <https://www.jonbell.net/preprint/icse21-flakeflagger.pdf>
- [11] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "Empirically Evaluating Flaky Test Detection Techniques Combining Test Case Rerunning and Machine Learning Models," *Empirical Software Engineering*, 2023. <https://link.springer.com/article/10.1007/s10664-023-10307-w>
- [12] A. Shi, A. Gyori, O. Legunsen, and D. Marinov, "Detecting Assumptions on Deterministic Implementations of Non-deterministic Specifications," in *ICST*, 2016. <https://mir.cs.illinois.edu/awshi2/publications/FSEDEMO2016.pdf>